
Informe del Proyecto

Sistema de Gestión de Reservas de Tutorías

Proyecto Final de Desarrollo Orientado a Objetos (Grupo 16)

Gerhec Ain Parra Gavilán Diego Felipe Fuentes Conejeros
Martín Alonso Labarca Rojas

Julio 2026

1. Idea general del flujo

Pensamos el programa como una aplicación de escritorio para que un administrador pueda gestionar tutores, estudiantes y reservas de clases, sin necesidad de una base de datos. Mientras el programa está abierto, todo se mantiene en memoria, y al arrancar se cargan algunos datos de ejemplo (tutores, estudiantes y también un puñado de reservas ya agendadas entre ellos) para que las cuatro pestañas de la aplicación, incluyendo Reservas y Calendario, empiecen pobladas en lugar de vacías la primera vez que se abre.

El funcionamiento interno sigue siempre el mismo recorrido, sin importar qué esté haciendo el usuario en ese momento.

Primero, el usuario interactúa con una pantalla, lo que llamamos la Vista. La Vista le entrega esos datos a un Controlador, que a su vez se los pasa a un Servicio del Modelo, que es quien realmente valida las reglas del negocio y guarda la información. Si ese Servicio agrega, edita o borra un dato, avisa automáticamente a las partes de la interfaz que dependen de esa información, para que se actualicen solas sin que nadie tenga que acordarse de refrescar nada a mano.

Esa secuencia no es casualidad: refleja cómo dividimos las responsabilidades del proyecto entre nosotros. La Vista solo dibuja pantallas y recibe clics, el Controlador únicamente traduce lo que el usuario pidió en una llamada al Modelo, y el Modelo es el único que conoce las reglas del negocio, como que un tutor no puede tener dos horarios que se crucen entre sí.

2. El Modelo

El Modelo es el núcleo del sistema: ahí viven los datos y las reglas de negocio, y es la única capa que no depende para nada de la interfaz gráfica. Más adelante, en la sección sobre por qué elegimos MVC como base, explicamos por qué esa independencia nos resultó tan útil a la hora de comprobar que todo funcionaba.

2.1. Entidades

Dentro del Modelo, las entidades representan los datos reales del negocio.

Estudiante guarda los datos de contacto de un alumno junto con un identificador único, y no se puede crear ni modificar con datos vacíos. Cuando se “elimina” un estudiante, en realidad se lo marca como inactivo en lugar de borrarlo, para no perder el historial de reservas que ya tenía.

Tutor tiene una forma parecida, pero además guarda las materias que dicta y los bloques de horario en los que está disponible. Se protege a sí mismo para que nunca queden dos materias repetidas ni dos horarios que se superpongan, sin depender de que quien lo use se acuerde de validar antes de agregar algo.

Materia es más simple: nombre, tarifa por hora y cupo máximo, con el detalle de que el nombre no se puede cambiar una vez creada, porque otras partes del sistema la identifican justamente por ese nombre.

BloqueHorario representa un rango de tiempo: un día, una hora de inicio y una hora de fin. Su función más importante es decidir si dos bloques se cruzan, algo que terminamos reutilizando en varios lugares distintos, como revisar la disponibilidad de un tutor, evitar que un estudiante reserve dos clases que chocan entre sí, contar cupos y dibujar el calendario.

Reserva une a un tutor, un estudiante, una materia y un bloque de horario, guardando si está activa, cancelada o completada. Una reserva nunca se borra, solo cambia de estado, para conservar el historial completo de lo que fue pasando.

2.2. Enumeraciones y excepciones

Para no depender de texto libre en campos como el día de la semana o el estado de una reserva, usamos enumeraciones cerradas de valores válidos, lo que evita errores de tipeo y hace que el propio compilador avise si alguna vez olvidamos contemplar algún caso. También definimos una lista de los avisos que el Modelo puede emitir hacia la Vista, como tutor agregado o reserva creada, que explicamos junto con el Observer más abajo.

Para los errores, en vez de usar un único tipo de excepción genérica, definimos varios tipos distintos: datos inválidos, conflicto de horario, cupo máximo alcanzado, tutor no disponible y elemento no encontrado. Esto le permite al Controlador reaccionar de forma distinta según qué salió mal, y mostrarle al usuario un mensaje que realmente tenga sentido para el error que ocurrió, en vez de un genérico “algo falló”.

2.3. Observer

El Observer es el mecanismo que mantiene a la Vista al día sin que el Controlador tenga que acordarse de refrescar nada por su cuenta.

La idea es simple: cada servicio puede tener una lista de observadores registrados, y cada vez que agrega, edita o elimina un dato, recorre esa lista y les avisa a todos. Los paneles de la Vista son quienes se registran como observadores, así que apenas el Modelo cambia algo, la tabla correspondiente se actualiza sola, sin que nadie tenga que llamarla explícitamente.

2.4. Strategy

El Strategy resuelve el problema de buscar tutores según distintos criterios sin terminar con un único método gigante lleno de condiciones anidadas. Cada criterio posible (materia, horario, tarifa, disponibilidad de cupo) vive en su propia clase, y esas piezas se pueden combinar según haga falta.

En el flujo real de agendar una clase terminamos usando una sola estrategia, la de disponibilidad de cupo, porque por sí sola ya cubre materia, horario y cupo al mismo tiempo. Las otras quedan disponibles para futuros flujos de búsqueda que se quieran agregar, sin tener que tocar el código que ya funciona.

2.5. Servicios

Los servicios son quienes realmente aplican las reglas del negocio y guardan los datos en memoria.

EstudianteService agrega, modifica y elimina lógicamente a los estudiantes, evitando que dos estudiantes activos compartan el mismo correo. **TutorService** hace lo mismo para tutores, y además centraliza el agregar materias y bloques de horario a un tutor ya existente, delegando esa validación al propio objeto Tutor.

ReservaService es el que tiene más reglas de todos, porque agendar una clase depende de varias condiciones a la vez: que el tutor esté disponible en ese horario, que el estudiante no tenga otra clase que choque, y que no se haya llegado al cupo máximo de esa materia en ese bloque. Solo si las tres condiciones se cumplen se crea la reserva.

CalendarioService no agrega reglas nuevas, solo reorganiza las reservas que ya existen para que sean fáciles de mostrar en una vista semanal. Y **BuscadorDisponibilidad** es quien conecta el Strategy con los datos reales, aplicando en cadena las estrategias que se le indiquen sobre la lista de tutores disponibles.

3. Los Controladores

A los controladores los pensamos intencionalmente delgados: casi no tienen lógica propia, solo reciben datos de la Vista, llaman al servicio correspondiente del Modelo, y traducen el resultado o el error en algo que la Vista pueda mostrarle al usuario.

EstudianteController y **TutorController** manejan el alta, la edición y la baja de estudiantes y tutores respectivamente, y el de tutores además puede mostrar sus propios mensajes de confirmación y error directamente en pantalla.

ReservaController conecta el servicio de reservas con el buscador de tutores disponibles: primero permite buscar tutores según materia y horario, y luego agendar la clase con el tutor elegido, traduciendo cada tipo de error de negocio (sin cupo, horario ocupado, tutor no disponible) en un mensaje distinto para el usuario.

CalendarioController es el más simple de todos: solo delega en el servicio de calendario, sin agregar lógica propia. Existe sobre todo para mantener la regla de que la Vista nunca hable directamente con el Modelo.

4. La Vista

Esta capa contiene todas las pantallas de la aplicación, divididas en paneles, que van dentro de la ventana principal, y diálogos, que son ventanas emergentes para agregar o editar datos.

MainFrame es el punto de arranque de todo: arma los servicios, los controladores y las pantallas, y conecta cada panel como observador del servicio que le corresponde. Es la única clase que conoce y crea todas las piezas del sistema, mientras que el resto solo recibe, a través de su constructor, las piezas que necesita para funcionar. A esta técnica se la conoce como inyección de dependencias.

PanelTutores, **PanelEstudiantes** y **PanelReservas** siguen la misma receta: una tabla con los datos y botones para agregar, editar y eliminar, y los tres se actualizan solos cuando el Modelo cambia, gracias al Observer. En el caso de **PanelReservas**, la tabla suma dos columnas calculadas para cada clase agendada, la tarifa por hora de la materia y el costo total de la sesión (tarifa por hora multiplicada por la duración del bloque horario), para que el costo de cada reserva quede visible sin tener que ir a revisarlo por separado en la ficha del tutor.

Para que los botones se vean iguales en toda la aplicación, sin repetir el mismo bloque de código en cada panel y diálogo, creamos una pequeña clase utilitaria, **botonesUI**, con un único método estático que recibe un botón y le aplica un estilo oscuro consistente (fondo negro, texto blanco y un resaltado sutil al pasar el mouse). Todos los paneles y diálogos que tienen botones llaman a ese método al crearlos, así que cualquier ajuste futuro a la apariencia de los botones se hace en un solo lugar en vez de en cada pantalla por separado.

PanelCalendario y **CalendarioCanvas** muestran la vista semanal de reservas, separadas en dos clases porque dibujar el calendario a mano es una tarea bastante distinta de manejar los controles de filtro, así que cada una se concentra en lo suyo.

DialogoTutor es el formulario más completo, porque además de los datos de contacto permite agregar materias y bloques de horario al tutor, y el mismo diálogo sirve tanto para crear un tutor nuevo como para editar uno existente. **DialogoEstudiante** es una versión mucho más simple del mismo patrón, con un formulario de cuatro campos.

DialogoReserva implementa el flujo de dos pasos para agendar una clase: primero buscar tutores disponibles según materia y horario, y solo después de ver los resultados, agendar la clase con el tutor elegido. Separamos la búsqueda del agendamiento en dos pasos justamente para evitar que el usuario reserve a ciegas sin saber antes si hay cupo.

5. Por qué elegimos estos patrones de diseño

5.1. MVC (Modelo-Vista-Controlador)

Elegimos MVC porque separa el proyecto en tres capas con una sola responsabilidad cada una: el Modelo tiene las reglas del negocio y los datos, la Vista solo dibuja pantallas, y el Controlador es el puente entre ambas. Si mañana cambia una regla de negocio, el cambio ocurre solo en el Modelo; si cambia el diseño visual, el cambio ocurre solo en la Vista. Sin esta separación, ambos tipos de cambio habrían quedado mezclados en las mismas clases, y modificar uno arriesgaría romper el otro sin que nadie lo notara a tiempo.

También pesó un argumento de dirección de dependencias: el Modelo no depende de la Vista ni del Controlador, pero estos dos sí dependen del Modelo. Eso significa que el Modelo se podría reutilizar tal cual con una interfaz completamente distinta, por ejemplo una versión web o una aplicación de consola, sin tocarle una sola línea.

Y, en la práctica, MVC nos facilitó dividir el trabajo en equipo sin pisarnos: mientras una persona escribía y probaba las reglas de negocio, otra podía construir las pantallas asumiendo que esas reglas ya estaban listas y funcionando, sin tener que esperar a que la interfaz gráfica estuviera terminada, ni al revés.

5.2. Observer

Lo usamos para que la Vista se mantuviera al día sin que el Controlador tuviera que acordarse de refrescar cada tabla manualmente. Sin este patrón, la única forma de que un panel se enterara de un cambio sería que el propio servicio conociera y llamara directamente a ese panel, lo cual habría roto la separación entre Modelo y Vista y hubiera obligado al servicio a conocer a todos los paneles interesados en sus datos.

Con Observer, el servicio solo conoce una interfaz genérica de “observador”, nunca una pantalla concreta. Además, agregar un panel nuevo que necesite enterarse de los mismos cambios es tan simple como registrarlo, sin tocar el servicio ni los paneles que ya existían.

5.3. Strategy

Lo usamos para la búsqueda de tutores porque los criterios de búsqueda se pueden pedir en distintas combinaciones. En vez de un método con muchas condiciones anidadas, cada criterio vive en su propia clase, y armamos la búsqueda juntando solo las que hacían falta.

Esto nos permite agregar un criterio nuevo en el futuro, por ejemplo buscar por calificación del tutor, sin modificar el resto del código ya existente, y también probar cada criterio de forma aislada, sin necesitar el resto del sistema para verificar que funciona bien.

6. Por qué era más práctico validar el proyecto con MVC como base

La razón principal es que el Modelo está escrito de forma completamente independiente de la interfaz gráfica.

Eso tiene una consecuencia directa a la hora de comprobar que el proyecto funciona: las reglas de negocio más importantes se pueden probar llamando directamente a los métodos del Modelo, sin abrir ninguna ventana ni hacer clic en nada. Entre esas reglas están que un tutor no dicte materia repetida, que sus horarios no se crucen, que un estudiante no reserve dos clases que chocan, y que no se pase el cupo máximo. De hecho, el proyecto ya cuenta con pruebas automatizadas escritas con JUnit que validan justamente esas reglas.

Si no hubiéramos separado estas capas, y esas mismas reglas hubieran quedado escritas dentro de los botones de la interfaz, la única forma de comprobar que una regla funciona habría sido ejecutar la aplicación completa, llenar formularios y hacer clic una y otra vez. Cualquier error de lógica habría quedado escondido detrás de la interfaz gráfica, y habría sido mucho más lento encontrarlo.

Esto también se reflejó en cómo organizamos el trabajo en equipo: tratamos el Modelo y los Controladores como una base ya estabilizada y probada, y concentramos el trabajo posterior en la Vista, construyendo pantallas sobre reglas que ya sabíamos que funcionaban.

7. Autocrítica: qué quedó pendiente y por qué

Llevamos este proyecto en paralelo con otros ramos del semestre, y en algún punto tuvimos que priorizar: optamos por dejar cerrado y funcionando el flujo completo de punta a punta, es decir MVC, Observer, Strategy y una interfaz gráfica usable, antes que pulir cada detalle interno. Lo hicimos bajo la lógica de que un sistema completo pero con inconsistencias menores es más valioso, y más fácil de corregir después, que un sistema perfecto en una sola capa pero incompleto en las demás. Eso significó que, revisando el código ya terminado, quedaron algunas asimetrías que un ciclo de refactor con más tiempo debería resolver.

La primera tiene que ver con validaciones desiguales entre entidades: Estudiante valida en su propia clase que ningún dato de contacto venga vacío, pero Tutor no hace esa misma validación por sí solo, sino que depende de que quien lo cree pase por el servicio correspondiente. Lo correcto habría sido que ambas entidades protegieran sus propios datos de la misma forma, sin depender de que la capa de servicio se acordara de hacerlo.

Algo parecido pasa con las reglas de negocio entre servicios: el servicio de estudiantes impide que dos estudiantes activos compartan el mismo correo, pero el servicio de tutores no tiene esa misma regla, así que hoy es posible registrar dos tutores con el mismo correo sin que el sistema se queje.

También nos dimos cuenta de que la modificación de una reserva no vuelve a aplicar las validaciones que sí se aplican al crearla: crear una reserva revisa disponibilidad del tutor, choques de horario y cupo máximo, pero modificarla simplemente reemplaza el horario sin repetir esas revisiones. Hoy ese método no está conectado a ningún botón de la interfaz, así que en la práctica no se puede disparar este caso.

De forma parecida, editar un tutor no termina de guardar los cambios en sus materias y horarios: el formulario permite agregar o quitar materias y bloques de horario en pantalla, pero esos cambios no se guardan de verdad en el Modelo al presionar “Guardar”; solo se guardan los datos de contacto. Es una inconsistencia real entre lo que la interfaz sugiere que hace y lo que efectivamente queda almacenado, y es probablemente el punto que más nos gustaría corregir con más tiempo.

Por último, el manejo de errores no quedó del todo parejo entre controladores: algunos métodos devuelven el mensaje específico del error y otros devuelven uno genérico; algunos atrapan cualquier excepción y otros dejan que se propague sin control. Esto no llegó a causarnos problemas en las pruebas manuales que hicimos, porque el flujo normal de la interfaz no genera esos casos, pero reconocemos que es un punto frágil de cara al proyecto.

Hay además algunas ideas que evaluamos y descartamos por falta de tiempo, no porque nos parecieran malas: construir un tutor completo, con datos, materias y horarios, en un solo paso en vez de ir agregando piezas después; reemplazar los mensajes de depuración por consola por un sistema de registro más serio; y unificar de una vez el estilo de manejo de errores en todos los controladores.

En resumen, creemos que el sistema cumple el flujo completo que pedía el enunciado, que las reglas de negocio principales están bien probadas, tanto manualmente como con pruebas automatizadas, y que las asimetrías que describimos arriba son justamente el tipo de detalle que normalmente se atraparía en una revisión de código más pausada. Por falta

de tiempo, frente a otros ramos y entregas en paralelo, quedaron para una futura iteración en vez de para esta entrega final.